



Stop Being the Dag Bottleneck



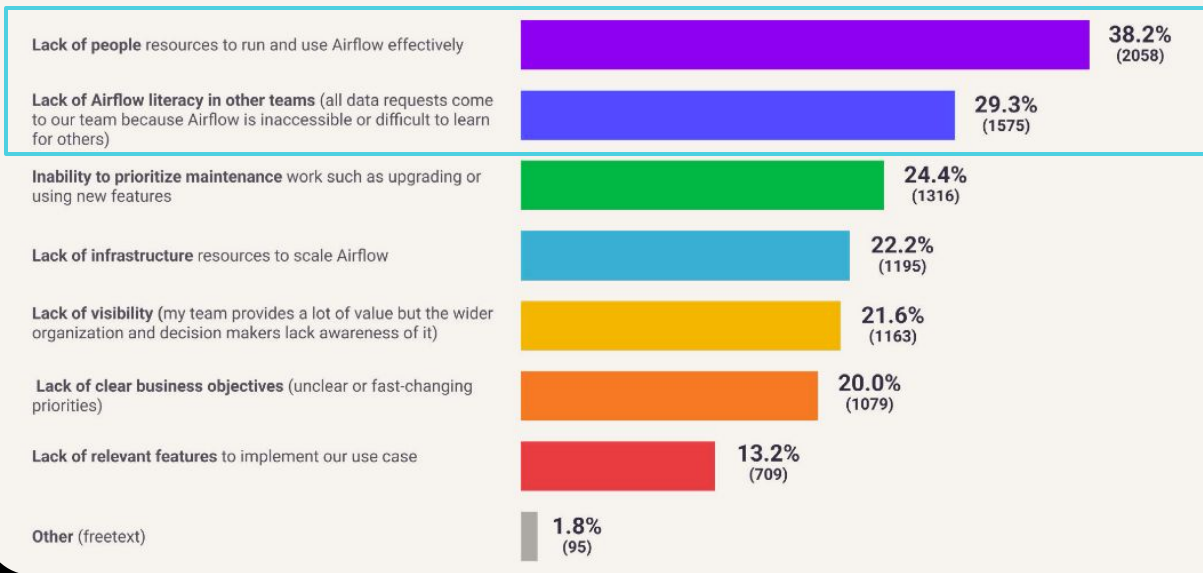
Yetunde Dada, Senior Director of Product Management

Jeremy Beard, Principal Software Engineer

Why have we focused on this problem?



What are the main challenges that you or your team face when trying to deliver value with Airflow to your business? (n=5384, multiple answers possible)



Why have we focused on this problem?



--- Effort on you: High

You make all the Dags

Whenever work needs to be orchestrated then you will be tagged to create the Airflow Dag.

--- Effort on you: Medium

You provide a path

They use interfaces in tools like Lakeflow Jobs or dbt Cloud and you use Airflow to orchestrate their work.

--- Effort on you: Low(ish)

They make all the Dags

You build or use an abstraction on top of Airflow that allows teams to create Airflow Dags.

Teams who have built or use their own abstractions

idealo

Booking.com

Apple

Wix

McDonald's

Credit Karma

Datacamp

Let's look at a different approach



We maintain Dag Factory and it exposes Airflow's full API via YAML.

```
default:
  default_args:
    start_date: 2024-11-11

basic_example_dag:
  default_args:
    owner: "custom_owner"
  description: "this is an example dag"
  schedule: "0 3 * * *"
  render_template_as_native_obj: True
  catchup: false
  tasks:
    - task_id: "task_1"
      operator: airflow.operators.bash.BashOperator
      bash_command: "echo 1"
```

Dag Factory is great for Data Engineers



It's really useful when:

- You're comfortable with Airflow and want all of Airflow's flexibility
- You spend a lot of time generating similar Dags

Dag Factory (*somewhat*) solves for Python literacy but not Airflow literacy

But we learnt a few things



- Lowering the barrier to entry with Airflow means hiding Airflow
- Platform teams still enjoy building abstractions for the teams they help
- The abstraction is not ours to design, but rather we should give you the tools to do it yourselves



So we built **Blueprint**

Blueprint is an abstraction framework



- Blueprint enables you to abstract Airflow Dags the way you want
- You build your own abstractions on the framework
 - For your organization
 - For your terminology
 - For your systems
 - For your data
- Anyone can then compose Dags with simple YAML configuration
 - Or build your own interface for what best suits your users

Defining the abstractions



- The abstractions are known as blueprints:
 - Building blocks that do something your Dag developers will repeatedly do
 - Specific enough that they are easy to understand and configure
 - Generic enough that they are reusable across many Dags

What is a blueprint?



- A blueprint is a Python class that generates an Airflow task group
- Driven by configuration fields you define, validated by Pydantic

Defining the abstractions – example



```
class DatabricksJobConfig(BaseModel):
    job_id: int = Field(description="Databricks job to trigger")
    target_table: str = Field(description="e.g. analytics.orders")

class DatabricksJob(Blueprint[DatabricksJobConfig]):
    def render(self, config: DatabricksJobConfig) -> DatabricksRunNowOperator:
        return DatabricksRunNowOperator(
            task_id=self.step_id,
            databricks_conn_id="databricks_prod",
            job_id=config.job_id,
            notebook_params={
                "target_table": config.target_table,
                "run_date": "{{ ds }}",
            },
        )
```

Composing the Dags



- Anyone can then compose a Dag from the blueprints, in YAML
- The YAML Dag has a list of steps
- Each step invokes one blueprint with the configurations for that step
- Steps can depend on other steps
- Airflow parses as a Dag of task groups with dependencies between them

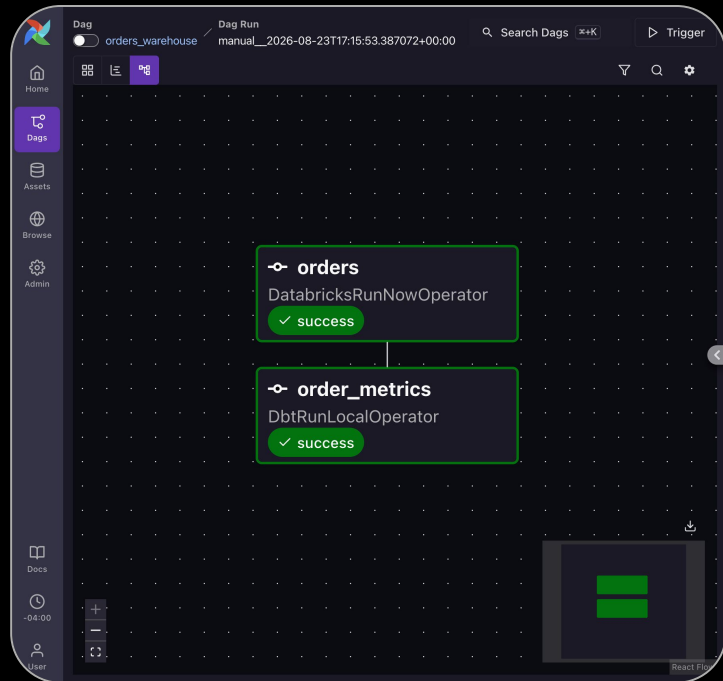
Composing the Dags - example



```
dag_id: orders_warehouse
schedule: "@daily"

steps:
  orders:
    blueprint: databricks_job
    job_id: 421
    target_table: analytics.orders

  order_metrics:
    blueprint: dbt_run
    depends_on: [orders]
    select: marts.orders+
```



Bringing it into Airflow



- Add your blueprint and YAML files to your Airflow dags dir
- And a single loader file to tell Airflow to use Blueprint to build the Dags

```
# dags/loader.py

from blueprint import build_all_airflow_dags

build_all_airflow_dags()
```

Blueprint is highly customizable



Versioned blueprints

Blueprint CLI

Advanced dependencies

Custom Dag args

Multiple loaders

Python Dag interop

Airflow UI visibility

Custom validations

Variables

Programmatic Dags

Dag post-processing

idealo's platform team stopped being the bottleneck



106 → 6

Lines to stand up a new pipeline.
They've reduced boilerplate code.

3h → 10m

To onboard a new team. A 3-hour
Airflow doc has become a short
form.

4

Teams onboarded so far, several
staffed by engineers who don't
write Python.

"Blueprint works at both ends of our spectrum. Engineers who are strong in their own stack but have never written Python use it to get running fast. And analysts who used to struggle just to set up a job find it easier too."

Ola Zajac • Product Owner at idealo

Getting started



- Blueprint repo: github.com/astronomer/blueprint
 - Includes examples you can run on local Airflow or on Astro
- Add the Blueprint skill to your agent: github.com/astronomer/agents



Questions?

yetunde.dada@astronomer.io

jeremy@astronomer.io

The 2026 Apache Airflow® Survey is Live!

- Takes just 7 minutes
- Get a free AI Orchestration Fundamentals Certification Exam or DAG Authoring Certification (normally \$150 each)

